CYBERSIREN — SVC-04 EMAIL METADATA ANALYSIS SERVICE — DETAILED SPECIFICATION

Document: SVC04-SPEC-v1.0 **Date:** 2026-03-13 **Status:** Research-Informed Design — Ready for Implementation **Parent:** ARCH-SPEC-v2.1 **Classification:** Internal / Graduation Project

DETERMINISTIC	Rule-based / protocol-mandated logic	**ML INFERENCE**	Trained model produces score	**ENSEMBLE**	Combines sub-block outputs	**ENRICHMENT**	External data lookups

1. SERVICE OVERVIEW

EMAIL METADATA ANALYSIS SERVICE — Python (FastAPI + ONNX Runtime) + Go (Kafka consumer wrapper) **SVC-04**

Responsibility: Assess email authenticity and sender trustworthiness from email header metadata and domain intelligence. Replaces the original heuristic-only Header Analysis Service with a **multi-block inference pipeline** that fuses deterministic protocol checks, ML-based domain risk scoring, embedding-based typosquatting detection, and routing anomaly classification into a single explainable header_risk_score (0–100).

Design rationale: Recent literature consistently demonstrates that hybrid approaches combining handcrafted header features with gradient-boosted ensembles substantially outperform pure heuristic systems. A 2025 study on multi-domain phishing feature analysis found that Random Forest achieved 99.85% accuracy on email header features when feature selection (RFE + variance thresholding) was applied, and that combining ML outputs with weighted heuristic rules delivered over 95% average accuracy on real-world datasets (Jadhav & Chandre, ETASR Vol. 15, 2025). The MeAJOR Corpus study (Afonso et al., arXiv:2507.17978, 2025) identifies header features — including received hop sequences, From/Return-Path/Reply-To discrepancies, and SPF/DKIM/DMARC results — as strong discriminative signals when combined with domain-level features. The EXPLICATE framework (Lim et al., arXiv:2503.20796, 2025) achieves 98.4% accuracy with SHAP/LIME explainability on domain-specific features.

Architecture: Five analysis blocks execute sequentially (Blocks 1–2 may run concurrently). Block outputs feed into a final ensemble that produces the composite score. Every block emits structured signals that map to rule_hits rows for full explainability. The service is deployed as a FastAPI HTTP microservice with a Go-based Kafka consumer wrapper (same pattern as SVC-06 NLP Analysis).

Kafka input: analysis.headers
Kafka output: scores.header
Timeout: 8s total. On timeout: header_risk_score = 50.

2. HIGH-LEVEL PIPELINE

```
analysis.headers message
    │
    ├──────────────────────────────┐
    │                              │
    ▼                              ▼
┌──────────────────┐     ┌──────────────────────┐
│  BLOCK 1         │     │  BLOCK 2             │
│  Authentication  │     │  Domain Intelligence │
│  Verification    │     │  (WHOIS + DNS + TI)  │
│  [DETERMINISTIC] │     │  [ENRICHMENT → ML]   │
│                  │     │                      │
│  → auth_sub_score │     │  → domain_risk_score │
│    (0-100)       │     │    (0-100)           │
```

```
└──────────────────┘     └──────────────────────┘
         │                         │
         ▼                         ▼
┌──────────────────┐     ┌──────────────────────┐
│  BLOCK 3         │     │  BLOCK 4             │
│  Typosquatting &  │     │  Routing & Structural │
│  Impersonation   │     │  Anomaly Detection   │
│  Detection       │     │  [ML]                │
│  [ML]            │     │                      │
│                  │     │  → routing_anomaly_score │
│  → typosquat_score │     │    (0–100)           │
│    (0–100)       │     │                      │
└──────────────────┘     └──────────────────────┘
         │                         │
         └─────────────────────────┘
                   ▼
          ┌──────────────────┐
          │  BLOCK 5         │
          │  Header Risk     │
          │  Ensemble        │
          │  [ENSEMBLE / ML] │
          │                  │
          │  → header_risk_score│
          │    (0–100)       │
          └──────────────────┘
                   │
                   ▼
            scores.header
```

Concurrency model: Blocks 1 and 2 execute concurrently (Block 2 performs network I/O for enrichment). Block 3 depends on Block 2's domain data. Block 4 is independent of Blocks 2–3 and may execute concurrently with them. Block 5 gathers all sub-scores and runs the final ensemble. Total target latency: < 4s p95 (excluding enrichment cache misses).

3. BLOCK 1 — AUTHENTICATION VERIFICATION

Authentication Verification Block

DETERMINISTIC

Function: Evaluates email authentication protocol results (SPF, DKIM, DMARC, ARC) extracted from headers by the Email Parser (SVC-02). Detects envelope/header identity mismatches. Produces auth_sub_score (0–100, where 100 = maximum risk). This block is deterministic — no ML model, no external calls. All logic operates on header fields already present in the analysis.headers Kafka message.

3.1 Input Fields Consumed

Field	Source	Type	Description
`auth_spf`	analysis.headers	enum	`"pass"` | `"fail"` | `"softfail"` | `"neutral"` | `"none"` | `"temperror"` | `"permerror"`
`auth_dkim`	analysis.headers	enum	`"pass"` | `"fail"` | `"none"` | `"temperror"` | `"permerror"`
`auth_dmarc`	analysis.headers	enum	`"pass"` | `"fail"` | `"none"` | `"bestguesspass"`
`auth_arc`	analysis.headers	enum	`"pass"` | `"fail"` | `"none"`
`sender_email`	analysis.headers	string	From: header address
`reply_to_email`	analysis.headers	string|null	Reply-To: header address
`return_path`	analysis.headers	string|null	Return-Path: envelope sender
`sender_domain`	analysis.headers	string	Domain extracted from sender_email

3.2 Signal Evaluation Rules

Each signal fires independently and contributes a weighted point value to auth_sub_score. Points are additive (capped at 100). Weights are stored in the rules table (target = "auth") and are adjustable without code changes.

Signal ID	Condition	Default Weight	Rationale
`AUTH-SPF-FAIL`	`auth_spf = "fail"`	25	Hard SPF failure means the sending IP is not authorized by the domain's SPF record. Strong spoofing indicator.
`AUTH-SPF-SOFTFAIL`	`auth_spf = "softfail"`	15	SPF softfail (~all) — sender IP not explicitly authorized but domain uses permissive policy. Weaker signal than hard fail.
`AUTH-SPF-NONE`	`auth_spf = "none"`	10	Domain publishes no SPF record at all. Legitimate orgs overwhelmingly publish SPF since Google/Yahoo mandates (2024).
`AUTH-DKIM-FAIL`	`auth_dkim = "fail"`	30	DKIM signature validation failed — message was modified in transit or signature is forged. High-confidence tampering signal.
`AUTH-DKIM-NONE`	`auth_dkim = "none"`	10	No DKIM signature present. Increasingly suspicious post-2024 bulk sender mandates.
`AUTH-DMARC-FAIL`	`auth_dmarc = "fail"`	35	DMARC alignment failure — SPF/DKIM domains do not align with the From: domain. Strong phishing signal. Highest weight because DMARC integrates both SPF and DKIM alignment.

`AUTH-DMARC-NONE`	`auth_dmarc = "none"`	10	Domain publishes no DMARC record. Moderate signal — many small domains still lack DMARC.
`AUTH-ARC-FAIL`	`auth_arc = "fail"`	10	ARC chain validation failure. ARC preserves authentication results across forwarding; failure suggests chain tampering.
`AUTH-IDENTITY-MISMATCH-FROM-REPLYTO`	`domain(sender_email) ≠ domain(reply_to_email)` AND `reply_to_email` is not null	20	From: and Reply-To: domains differ. Common in phishing — attacker spoofs a brand in From: but replies go to an attacker-controlled address.
`AUTH-IDENTITY-MISMATCH-FROM-RETURNPATH`	`domain(sender_email) ≠ domain(return_path)` AND `return_path` is not null	15	From: and Return-Path: domain mismatch. Weaker than Reply-To mismatch (some legitimate services use different bounce domains), but still a relevant signal.
`AUTH-TRIPLE-FAIL`	`auth_spf ∈ {fail, softfail, none}` AND `auth_dkim ∈ {fail, none}` AND `auth_dmarc ∈ {fail, none}`	+15 (bonus)	Compound signal: all three authentication mechanisms absent or failing simultaneously. Additional penalty on top of individual signal scores.

3.3 Score Computation

```
auth_sub_score = min(100, sum(weight for each fired signal))
```

3.4 Output

Field	Type	Description
`auth_sub_score`	int 0–100	Composite authentication risk. 0 = all protocols pass and identities align. 100 = severe authentication failures.
`auth_signals`	object	`{ spf_result, dkim_result, dmarc_result, arc_result, from_replyto_match: bool, from_returnpath_match: bool }`
`auth_fired_rules`	array	List of `{ rule_id, rule_name, score_impact, match_detail }` for each fired signal

DB access: R rules (active auth rules, cached in-memory with 60s refresh). **W** rule_hits (one row per fired signal).
External calls: None. All input fields are pre-extracted by SVC-02.

4. BLOCK 2 — DOMAIN INTELLIGENCE

Domain Intelligence Block

ENRICHMENT | ML INFERENCE

Function: Assesses the trustworthiness of the sender's domain through enrichment data (WHOIS, DNS, TI) and an XGBoost classifier trained on domain-level features. Replaces the original heuristic-only domain age check with a proper learned model that captures non-linear relationships between registration patterns, DNS configuration, and phishing risk. The model is trained offline and deployed via ONNX Runtime.

> **Research basis:** Jadhav & Chandre (2025) demonstrate that gradient-boosted models (XGBoost, CatBoost) trained on email header features including domain properties achieve 96–99% accuracy. The PREDATOR system (Hao et al., IEEE S&P 2016) shows that time-of-registration features alone can predict malicious domains days before blacklist inclusion. The PhiUSIIL framework (Prasad & Chandra, Computers & Security 2024) combines domain features with incremental learning for adaptive detection. Afonso et al. (MeAJOR Corpus, arXiv:2507.17978, 2025) identify domain age, DNS record characteristics, and TLD patterns as high-importance engineered features for ML-based phishing classification.

4.1 Enrichment Pipeline

For the sender domain (sender_domain from the Kafka message), collect the following data. Cache strategy: Redis key domain_intel:{sha256(domain)} with 12h TTL. On cache hit, skip enrichment and use cached feature vector directly.

Data Source	Fields Collected	Timeout	Fallback on Failure
WHOIS Lookup	`creation_date`, `expiry_date`, `updated_date`, `registrar`, `name_servers[]`, `registrant_country`, `dnssec` (signed/unsigned), `privacy_protected` (bool)	3s	Set WHOIS features to null → model uses domain-only features (partial inference)
DNS Records	`has_mx` (bool), `mx_count`, `has_spf_record` (bool), `spf_record_text`, `has_dmarc_record` (bool), `dmarc_policy` (none/quarantine/reject), `a_record_count`, `ns_count`, `has_caa` (bool)	2s	Set DNS features to null → partial inference
TI Database	Lookup `sender_domain` against `enriched_threats` table. Returns: `ti_matched` (bool), `ti_risk_score` (int|null), `ti_threat_type`, `ti_first_seen`, `ti_last_seen`, `ti_source_feed`	50ms (DB query)	Set TI features to null → proceed without TI signal
Originating IP Geolocation	Lookup `originating_ip` (from Kafka message) against GeoIP database (MaxMind GeoLite2 local DB). Returns: `ip_country`, `ip_asn`, `ip_org`	<1ms (local)	Set to null

4.2 Feature Engineering

Raw enrichment data is transformed into a fixed-width feature vector for XGBoost inference. All features are numeric or one-hot encoded categorical. Missing values are handled natively by XGBoost (no imputation required).

#	Feature Name	Type	Derivation
1	`domain_age_days`	int|null	`(now - creation_date).days`. Null if WHOIS unavailable.
2	`domain_expiry_days`	int|null	`(expiry_date - now).days`. Short-lived domains correlate with phishing campaigns.
3	`domain_lifespan_days`	int|null	`(expiry_date - creation_date).days`. 1-year registrations are more suspect than multi-year.
4	`days_since_updated`	int|null	`(now - updated_date).days`. Recently updated WHOIS can indicate domain

			preparation for attack.
5	`privacy_protected`	bool|null	WHOIS privacy service active. Phishing domains frequently use privacy protection.
6	`dnssec_enabled`	bool|null	DNSSEC signing active. Legitimate high-value domains are more likely to have DNSSEC.
7	`tld_risk_category`	int 0–3	Pre-computed TLD risk tier. 0 = low-risk (.gov, .edu, .mil), 1 = standard (.com, .org, .net), 2 = elevated (.xyz, .top, .info, .click, .work), 3 = high-risk (newly delegated/abuse-prone gTLDs — maintained via TI data). Mapping table loaded from config.
8	`domain_length`	int	Character count of full domain (excl. TLD). Excessively long domains are suspicious.
9	`subdomain_depth`	int	Count of dots in domain minus TLD dots. Deep subdomains (e.g., login.secure.paypal.attacker.com) indicate abuse.
10	`has_digits_in_domain`	bool	Domain SLD contains digits (e.g., paypa1.com). Common in typosquatting.
11	`domain_entropy`	float	Shannon entropy of domain characters. Random-looking domains (DGA-generated) have high entropy.
12	`has_mx`	bool|null	Domain has MX records. Domains without MX that send email are suspicious.
13	`mx_count`	int|null	Number of MX records. Legitimate orgs typically have 1+ redundant MX.
14	`has_spf_record`	bool|null	Domain publishes SPF TXT record in DNS.
15	`has_dmarc_record`	bool|null	Domain publishes DMARC record.
16	`dmarc_policy_strength`	int 0–2|null	0 = none, 1 = quarantine, 2 = reject. Stricter policies indicate more mature domain.
17	`ns_count`	int|null	Number of NS records.
18	`registrar_risk_tier`	int 0–2|null	Pre-computed registrar reputation. 0 = established (e.g., GoDaddy, Namecheap, Google Domains), 1 = neutral, 2 = high-abuse (registrars flagged in TI data for hosting disproportionate phishing). Mapping table maintained by TI Sync.
19	`ti_matched`	bool	Sender domain found in `enriched_threats`.
20	`ti_risk_score`	int|null	Risk score from TI database if matched.
21	`ip_country_matches_domain_country`	bool|null	Geographic consistency between originating IP and registrant country.
22	`is_free_email_provider`	bool	Domain is in the free email provider list (gmail.com, yahoo.com, outlook.com, etc.). Maintained as a config list.
23	`has_caa`	bool|null	DNS CAA record exists (CA Authorization). Signals proactive security management.

4.3 Model Architecture

Property	Value
Algorithm	XGBoost (gradient boosted decision trees)

Implementation	Trained offline with `xgboost` Python library → exported to ONNX via `onnxmltools` → served via ONNX Runtime
Input	23-feature vector (see §4.2). Missing values handled natively (no imputation).
Output	Phishing probability `p ∈ [0.0, 1.0]` → scaled to `domain_risk_score = round(p × 100)`
Training data	WHOIS/DNS features extracted from: PhishTank verified URLs (domains), OpenPhish (domains), MeAJOR Corpus header features, Alexa Top 1M (legitimate domains). Target: ~50K domains (balanced phishing/legitimate).
Feature importance	SHAP values computed during training. Top features expected: `domain_age_days`, `ti_matched`, `tld_risk_category`, `dmarc_policy_strength`, `domain_entropy`.
Model size	~2–5 MB (ONNX). Inference: <10ms per domain.
Retraining cadence	Monthly. New domains from TI feeds + confirmed verdicts form the retraining set.

4.4 TI Short-Circuit

If ti_matched = true AND ti_risk_score ≥ 80 in the enriched_threats table: set domain_risk_score = 95 and skip model inference. This provides instant, authoritative scoring for known-bad domains and reduces latency. The short-circuit is logged as a fired rule DOMAIN-TI-BLOCKLIST.

4.5 Output

Field	Type	Description
`domain_risk_score`	int 0–100	ML-predicted or TI-derived domain risk. 0 = highly trusted domain. 100 = confirmed malicious.
`domain_features`	object	Full feature vector (for explainability and downstream use)
`domain_ti_matched`	bool	Whether domain was found in TI
`domain_age_days`	int|null	Exposed for campaign analysis
`is_free_email_provider`	bool	Flag for downstream contextual scoring
`enrichment_source`	enum	`"cache_hit"` | `"live_enrichment"` | `"ti_shortcircuit"` | `"partial"`

DB access: **R** enriched_threats (domain TI lookup, indexed on domain), **R** Redis: domain_intel:{hash}, **R** Redis: ti_domain:{domain}.
W Redis: domain_intel:{hash} (cache new enrichment results, TTL 12h).
External calls: WHOIS (rdap.org or python-whois), DNS resolver (system resolver or dnspython), GeoIP (local MaxMind DB).

5. BLOCK 3 — TYPOSQUATTING & IMPERSONATION DETECTION

Typosquatting & Impersonation Detection Block

ML INFERENCE

Function: Detects whether the sender domain is attempting to impersonate a known legitimate brand or organization through typosquatting (character substitution/transposition), homograph attacks (visually similar Unicode characters), or combosquatting (appending brand names to unrelated domains). Replaces the original simple edit-distance check with a multi-method ensemble using embeddings.

> **Research basis:** Piredda et al. ("Deepsquatting", AI*IA 2017) introduce ML-based similarity measures for typosquatting detection using n-gram representations. The Anvilogic CE-Typosquat-Detect model (HuggingFace, 2024) uses a CrossEncoder architecture for domain pair similarity scoring. Tchouangang et al. (Springer, 2025) apply Siamese neural networks for IDN homograph detection. The dnstwist tool (open source) provides comprehensive domain permutation generation algorithms. TypoSmart (arXiv:2502.20528, 2025) demonstrates that embedding-driven name analysis combined with metadata verification reduces false positives by 70% compared to edit-distance alone.

5.1 Brand Dictionary

A curated list of protected brand domains is maintained in a configuration file (JSON or DB table). Sources: Alexa/Tranco Top 500, phishing target brands from TI feeds (`enriched_threats.target_brand`), and manually added org-specific domains. Updated weekly via TI Sync. Estimated size: 500–2000 entries.

Field	Example	Description
`brand_domain`	`paypal.com`	Canonical domain of the brand
`brand_name`	`PayPal`	Display name (for reporting)
`brand_embedding`	*[float vector]*	Pre-computed character-level embedding of the domain name (see §5.2)
`brand_variants`	`["paypal.co.uk", "paypal.de"]`	Known legitimate alternate domains (whitelisted)

5.2 Detection Methods

Three complementary methods are applied to the sender domain against each brand in the dictionary. Candidate brands are pre-filtered by a fast approximate match (TF-IDF character trigram cosine similarity > 0.3) to avoid O(n) full comparisons.

Method	Algorithm	Output	Implementation
M1: Edit Distance	Damerau-Levenshtein distance between sender domain SLD and each candidate brand domain SLD. Normalized by max(len(a), len(b)).	`edit_sim` ∈ `[0.0, 1.0]` (1.0 = identical)	Python `jellyfish` or `rapidfuzz` library. O(n×m) per pair but fast for short domain strings.
M2: Character Embedding Similarity	Encode sender domain using a character-level embedding model (SentenceTransformers `all-MiniLM-L6-v2` with character n-gram input, or the Anvilogic/Embedder-typosquat-detect model). Compute cosine similarity between sender domain embedding and pre-computed brand embeddings.	`embed_sim` ∈ `[0.0, 1.0]`	Pre-compute brand embeddings at startup. Sender domain embedding computed at inference time. Cosine similarity via numpy. Model loaded via ONNX Runtime or sentence-transformers.
M3: Homoglyph / Visual Similarity	Map each character to its canonical form using a confusables table (Unicode NFKD normalization + custom confusables map derived from Unicode TR39). Compare normalized sender domain to brand domain. If normalized forms match but raw forms	`homoglyph_match: bool` `homoglyph_score: 0.0|0.85|1.0`	Python `confusable_homoglyphs` library + custom table covering common IDN attack characters (Cyrillic a/a, Latin/Greek o/o, etc.). Binary detection with a fixed score output.

	differ, flag as homograph attack.		

5.3 Score Fusion

```
For each candidate brand that passes pre-filter:
    edit_sim     = 1.0 - (damerau_levenshtein(sender_sld, brand_sld) / max(len(sender_sld),
len(brand_sld)))
    embed_sim    = cosine_similarity(sender_embedding, brand_embedding)
    homo_score   = 1.0 if homoglyph_match else 0.0

    # Exact match to brand or known variant → whitelist (score 0)
    if sender_domain ∈ brand_variants or sender_domain == brand_domain:
        continue  # skip this brand

    # Weighted fusion
    raw_similarity = 0.35 × edit_sim + 0.45 × embed_sim + 0.20 × homo_score

    # Only flag if similarity exceeds threshold (subject to calibration, default 0.70)
    if raw_similarity >= TYPOSQUAT_THRESHOLD:
        candidates.append({ brand, raw_similarity })

# Take highest-scoring candidate
if candidates:
    best = max(candidates, key=lambda c: c.raw_similarity)
    typosquat_score = round(best.raw_similarity × 100)
    typosquat_target = best.brand
else:
    typosquat_score = 0
    typosquat_target = null
```

5.4 Combosquatting Detection

Additionally checks if any brand name appears as a substring within the sender domain SLD (e.g., paypal-security-verify.com). If substring found and domain is not in the brand's whitelist → combosquat_detected = true, add +15 to typosquat_score (capped at 100).

5.5 Output

Field	Type	Description
`typosquat_score`	int 0–100	Impersonation risk score. 0 = no brand match. 100 = exact homoglyph of protected brand.
`typosquat_target`	string|null	Name of the impersonated brand, if any
`typosquat_distance`	int|null	Raw Damerau-Levenshtein distance to the target brand domain
`typosquat_method`	string|null	`"edit_distance"` | `"embedding"` | `"homoglyph"` | `"combosquat"` — which method contributed the highest signal
`combosquat_detected`	bool	Brand name embedded as substring in sender domain

DB access: R brand dictionary (in-memory, loaded at startup, refreshed every 6h from config/DB). **External calls:** None. Embedding model and brand embeddings are pre-loaded in memory. **Model deployment:** Embedding model (~90MB, MiniLM-L6 or similar) loaded once at service startup. Inference is <5ms per domain.

6. BLOCK 4 — ROUTING & STRUCTURAL ANOMALY DETECTION

Routing & Structural Anomaly Detection Block

ML INFERENCE

Function: Classifies structural anomalies in email headers — suspicious routing paths, timestamp inconsistencies, unusual mailer agents, and header manipulation patterns — using a lightweight XGBoost model trained on header structural features. This replaces the original heuristic-only structural checks with a model that can learn complex feature interactions.

> **Research basis:** The MeAJOR Corpus (arXiv:2507.17978, 2025) identifies Received header hop count, timestamp sequencing anomalies, and From/Return-Path discrepancies as high-discriminative header features. EvoMail (arXiv:2509.21129, 2025) highlights cross-modal fusion of header routing signals with domain features. The ChatSpamDetector study (arXiv:2402.18093, 2024) confirms that header analysis — particularly received chain validation and authentication result patterns — provides strong signals even without body text analysis.

6.1 Feature Engineering

#	Feature Name	Type	Derivation
1	`hop_count`	int	Count of `Received:` headers. Normal range: 3–8. Excessive hops (>12) may indicate open relay abuse.
2	`timestamp_drift_hours`	float	`abs(Date_header_timestamp - first_received_timestamp)` in hours. Large drift (>24h) indicates header manipulation or timezone spoofing.
3	`received_chain_monotonic`	bool	Whether timestamps in the Received chain are monotonically increasing (oldest → newest). Non-monotonic chains suggest header injection.
4	`received_chain_avg_delay_s`	float	Average time between consecutive Received hops. Very fast (<1s per hop) or very slow (>3600s per hop) is suspicious.
5	`received_chain_max_delay_s`	float	Maximum delay between any two consecutive hops. Single large gap may indicate queuing at a suspicious relay.
6	`unique_countries_in_chain`	int	Count of distinct countries in received chain IPs (via GeoIP). High geographic diversity (>4 countries) for simple emails is suspicious.
7	`first_hop_ip_private`	bool	First Received hop originates from a private/RFC1918 IP. Expected for corporate mail servers; suspicious for external senders claiming to be enterprise.
8	`mailer_agent_known`	bool	`X-Mailer` or `User-Agent` header matches a known legitimate client (Outlook, Thunderbird, Apple Mail, Gmail web, etc.). Unknown or absent MUA is a weak signal.
9	`mailer_agent_suspicious`	bool	`X-Mailer` matches known phishing toolkits (GoPhish, King Phisher, custom PHP mailers, etc.). Maintained as a config list.
10	`has_x_originating_ip`	bool	Presence of `X-Originating-IP` header (webmail origin indicator).
11	`precedence_header`	int 0–2	0 = absent, 1 = `bulk` or `list`, 2 = `junk`. Indicates self-identified mass mail.

12	`has_list_id`	bool	Presence of `List-Id` or `List-Unsubscribe` headers. Legitimate mailing lists have these; phishing generally does not.
13	`content_charset_suspicious`	bool	Email uses an unusual content charset (e.g., KOI8-R for English-language email, or mixed charsets) that doesn't match the detected body language.
14	`has_in_reply_to`	bool	Presence of `In-Reply-To` header indicating this is a reply. Phishing emails rarely carry valid reply threading.
15	`has_references`	bool	Presence of `References` header (email threading). Same reasoning as `In-Reply-To`.
16	`sender_domain_matches_received_chain`	bool	Sender domain appears in at least one Received: header's `by` or `from` field. Mismatch indicates the email did not originate from the claimed domain's infrastructure.
17	`originating_ip_in_sender_spf`	bool|null	If SPF record was fetched (Block 2), check if originating_ip is within the sender domain's SPF-authorized IP ranges. Requires parsing SPF record.

6.2 Model Architecture

Property	Value
Algorithm	XGBoost (gradient boosted decision trees)
Implementation	Trained offline → ONNX export → ONNX Runtime serving
Input	17-feature vector (see §6.1). Missing values handled natively.
Output	Anomaly probability `p ∈ [0.0, 1.0]` → `routing_anomaly_score = round(p × 100)`
Training data	Header structural features from: phishing_pot honeypot (phishing samples with full headers), MeAJOR Corpus header subset, Enron corpus (legitimate samples), SpamAssassin Public Corpus (mixed). Target: ~20K–50K emails (balanced).
Model size	~1–3 MB (ONNX). Inference: <5ms.
Retraining cadence	Monthly, synchronized with Block 2's model.

6.3 Output

Field	Type	Description
`routing_anomaly_score`	int 0–100	Structural anomaly risk. 0 = normal routing, standard headers. 100 = severe anomalies detected.
`routing_features`	object	Full feature vector for explainability
`routing_signals`	object	`{ hop_count, timestamp_drift_hours, chain_monotonic, mailer_agent, sender_domain_in_chain, unique_countries }`

DB access: None. All data derived from the Kafka message fields + GeoIP local DB.
External calls: None. GeoIP lookup is against a local MaxMind GeoLite2 database file.

7. BLOCK 5 — HEADER RISK ENSEMBLE

Header Risk Ensemble

ENSEMBLE

Function: Combines all sub-block outputs into a single header_risk_score (0–100). The ensemble uses a lightweight meta-model (XGBoost or logistic regression) that learns the optimal weighting of sub-scores rather than relying on hand-tuned weights. Individual sub-scores are always preserved alongside the composite for full explainability.

7.1 Meta-Model Input Features

#	Feature	Source Block	Type
1	`auth_sub_score`	Block 1	int 0–100
2	`domain_risk_score`	Block 2	int 0–100
3	`typosquat_score`	Block 3	int 0–100
4	`routing_anomaly_score`	Block 4	int 0–100
5	`is_free_email_provider`	Block 2	bool (contextual modifier)
6	`combosquat_detected`	Block 3	bool (high-signal binary)
7	`domain_ti_matched`	Block 2	bool (high-signal binary)

7.2 Meta-Model Architecture

Property	Value
Algorithm	XGBoost (max_depth=3, n_estimators=50) — intentionally shallow to prevent overfitting on 7 features
Training	Trained on labeled email datasets with per-block scores computed by Blocks 1–4. Labels: phishing (1) / legitimate (0) from merged corpora + analyst verdicts.
Output	`p ∈ [0.0, 1.0]` → `header_risk_score = round(p × 100)`
Fallback	If meta-model unavailable (cold start / model loading failure): use weighted average with default weights: `0.30 × auth + 0.30 × domain + 0.25 × typosquat + 0.15 × routing`
Explainability	SHAP values for each sub-score contribution are computed at inference time and included in the output. This allows the dashboard to show "the authentication failure contributed 45% to the header risk score."

7.3 Fallback Weighted Average (Cold Start / Bootstrap)

```
# Used when meta-model is not yet trained (initial deployment)
# Weights are preliminary defaults subject to calibration

header_risk_score = min(100, round(
    0.30 × auth_sub_score +
    0.30 × domain_risk_score +
    0.25 × typosquat_score +
    0.15 × routing_anomaly_score
))

# Override: if any single sub-score ≥ 90, floor the composite at 70
# Prevents a single critical signal from being averaged away
if max(auth_sub_score, domain_risk_score, typosquat_score, routing_anomaly_score) >= 90:
    header_risk_score = max(header_risk_score, 70)
```

8. KAFKA OUTPUT SCHEMA

Message schema — scores.header (Key: email_id · 6 partitions · 24h retention):

```
{
  email_id: uuid,
```

```
  org_id: int,
  component: "header",
  score: int 0-100,                    // header_risk_score (composite)
  auth_sub_score: int 0-100,
  domain_risk_score: int 0-100,
  typosquat_score: int 0-100,
  routing_anomaly_score: int 0-100,

  // Block 1 details
  auth_signals: {
    spf_result: string, dkim_result: string, dmarc_result: string, arc_result: string,
    from_replyto_match: bool, from_returnpath_match: bool
  },

  // Block 2 details
  domain_intel: {
    domain_age_days: int|null, tld_risk_category: int, is_free_provider: bool,
    ti_matched: bool, ti_threat_type: string|null, enrichment_source: string,
    has_spf_record: bool|null, has_dmarc_record: bool|null, dmarc_policy: string|null
  },

  // Block 3 details
  typosquat: {
    target_brand: string|null, distance: int|null, method: string|null,
    combosquat_detected: bool
  },

  // Block 4 details
  routing: {
    hop_count: int, timestamp_drift_hours: float, chain_monotonic: bool,
    sender_domain_in_chain: bool, unique_countries: int
  },

  // Explainability
  fired_rules: [{ rule_id: int, rule_name: string, rule_version: int, score_impact: int,
match_detail: json }],
  ensemble_shap: { auth: float, domain: float, typosquat: float, routing: float } | null,
  model_versions: { domain_xgb: string, routing_xgb: string, typosquat_embed: string, ensemble:
string },
  processing_time_ms: int
}
```

9. DATABASE ACCESS — COLUMN-LEVEL DETAIL

Operation	Table (Migration)	Columns Accessed	Direction	Details
Load auth rules	`rules` (003+008)	`id`, `name`, `version`, `target`, `condition`, `score_impact`, `enabled`, `rule_group_id`	READ	`SELECT * FROM rules WHERE target='auth' AND enabled=TRUE`. Cached in-memory, refreshed every 60s.
TI domain lookup	`enriched_threats` (001+002+003)	`id`, `domain`, `risk_score`, `threat_type`, `target_brand`, `first_seen`, `last_seen`, `source_feed`	READ	`SELECT * FROM enriched_threats WHERE domain=$sender_domain LIMIT 1`. Hot path via Redis cache `ti_domain:{domain}`.
Write fired rules	`rule_hits` (003)	`rule_id`, `entity_type='email'`, `entity_id`, `score_impact`, `match_detail` (JSONB), `created_at`	WRITE (INSERT)	One row per fired signal from Block 1. Batch insert for efficiency.

Redis access patterns:
R ti_domain:{sender_domain} — TI cache (populated by SVC-11 TI Feed Sync). TTL managed by SVC-11.
R/W domain_intel:{sha256(sender_domain)} — Domain enrichment cache. TTL 12h. Written by Block 2 after live enrichment. Read on cache hit to skip enrichment.

10. MODEL TRAINING & DATA PIPELINE

10.1 Training Data Sources

Dataset	Type	Size	Used For
MeAJOR Corpus	Multi-source phishing + legit	135K emails	All blocks (header features)
phishing_pot honeypot	Phishing .eml files with full headers	~10K (ongoing)	Blocks 2, 4 (real headers)
Enron Email Dataset	Legitimate corporate email	~500K	Blocks 2, 4 (negative class)
SpamAssassin Public Corpus	Mixed spam + ham	~6K	Validation set
Tranco Top 10K domains	Legitimate domains	10K	Block 2 (domain features, negative)
PhishTank verified phishing	Phishing URLs/domains	~50K domains	Block 2 (domain features, positive)
CyberSiren analyst verdicts	Production feedback loop	Growing	All blocks (retraining signal)

10.2 Training Pipeline

Offline batch process. Runs monthly or on-demand.

1. Extract header features from raw .eml files (Python email stdlib parser)
2. Run WHOIS/DNS enrichment for domain features (cached, rate-limited)
3. Label assignment: TI feed match = phishing, analyst verdict = ground truth, corpus labels = default
4. Feature engineering per §4.2, §6.1
5. Stratified train/val/test split (70/15/15)
6. Train XGBoost with Optuna hyperparameter search (50 trials)
7. Compute SHAP feature importances on validation set
8. Evaluate on test set: target F1 ≥ 0.95, FPR < 3%
9. Export to ONNX: onnxmltools.convert_xgboost(model)
10. Register model version in config. Deploy via container image update.

Tools: Python 3.11, xgboost, optuna, onnxmltools, shap, scikit-learn, pandas

11. DEPLOYMENT ARCHITECTURE

11.1 Service Structure

```
svc-04-email-metadata/
├── cmd/
│   └── consumer/          # Go Kafka consumer wrapper (same pattern as SVC-06)
│       └── main.go        # Consumes analysis.headers, calls Python inference service via HTTP,
publishes scores.header
├── inference/
│   ├── app.py             # FastAPI application entry point
│   ├── blocks/
│   │   ├── auth.py        # Block 1: Authentication Verification (deterministic)
│   │   ├── domain.py      # Block 2: Domain Intelligence (enrichment + XGBoost)
│   │   ├── typosquat.py   # Block 3: Typosquatting Detection (embeddings + edit distance)
│   │   ├── routing.py     # Block 4: Routing Anomaly Detection (XGBoost)
│   │   └── ensemble.py    # Block 5: Header Risk Ensemble (meta-model)
│   ├── enrichment/
│   │   ├── whois.py       # WHOIS lookup with caching
│   │   ├── dns.py         # DNS record queries
│   │   └── geoip.py       # MaxMind GeoLite2 lookups
│   ├── models/            # ONNX model files
│   │   ├── domain_xgb_v1.onnx
│   │   ├── routing_xgb_v1.onnx
│   │   ├── typosquat_embed_v1.onnx   # or sentence-transformers model
│   │   └── ensemble_xgb_v1.onnx
│   ├── config/
│   │   ├── brand_dictionary.json      # Protected brand domains
│   │   ├── free_providers.json        # Free email provider list
│   │   ├── tld_risk_tiers.json        # TLD risk categorization
│   │   ├── registrar_risk_tiers.json  # Registrar reputation map
│   │   └── suspicious_mailers.json    # Known phishing toolkit MUAs
│   └── requirements.txt
├── training/
│   ├── train_domain_model.py
│   ├── train_routing_model.py
│   ├── train_ensemble.py
│   ├── feature_engineering.py
│   └── evaluate.py
├── Dockerfile
└── docker-compose.yml
```

11.2 Runtime Dependencies

Dependency	Version	Purpose
Python	3.11+	ML inference runtime
FastAPI	0.110+	HTTP API for inference
onnxruntime	1.17+	Model serving (CPU)
sentence-transformers	2.7+	Embedding model for Block 3 (loaded once at startup)
rapidfuzz	3.6+	Fast Damerau-Levenshtein distance
confusable-homoglyphs	3.0+	Unicode homoglyph detection
python-whois	0.9+	WHOIS lookups (Block 2 enrichment)
dnspython	2.6+	DNS record queries
geoip2	4.8+	MaxMind GeoLite2 IP geolocation
numpy	1.26+	Numerical operations, cosine similarity
shap	0.45+	Runtime SHAP for ensemble explainability
Go 1.24	—	Kafka consumer wrapper

11.3 Resource Requirements

Resource	Minimum	Recommended	Notes
CPU	2 vCPU	4 vCPU	Embedding model benefits from multiple cores
Memory	1 GB	2 GB	~600 MB for embedding model + ONNX models + brand embeddings cache
Disk	500 MB	1 GB	Models + GeoIP DB (~70 MB) + brand dictionary

Startup time	15–30s	Model loading + brand embedding precomputation

12. ERROR HANDLING & RESILIENCE

Failure Mode	Affected Block	Behavior	Recovery
WHOIS lookup timeout (3s)	Block 2	Set WHOIS features to null. Model uses domain-only + DNS features (partial inference). Log warning.	No retry. Cached on next successful lookup for same domain.
DNS lookup timeout (2s)	Block 2	Set DNS features to null. Partial inference.	No retry.
ONNX model loading failure	Blocks 2, 4, 5	Service returns 503. Consumer pauses. Health check fails.	Kubernetes restart + readiness probe. Models are baked into container image.
Embedding model loading failure	Block 3	Fall back to edit-distance only (M1) for typosquatting. `typosquat_method = "edit_distance_fallback"`	Log critical. Container restart.
Ensemble model unavailable (cold start)	Block 5	Use weighted average fallback (§7.3).	Train and deploy ensemble model as soon as labeled data is available.
Total service timeout (8s)	All	Return `header_risk_score = 50` (neutral). `partial_analysis = true`.	Score Aggregator handles via 30s timeout fallback.
Redis connection failure	Blocks 2, 3	Skip cache, perform live enrichment every time. Higher latency but correct results.	Reconnect with backoff.
DB connection failure (rule_hits write)	Block 1	Buffer rule_hits in memory (max 100), retry on next message. Score computation unaffected.	3 retries, exponential backoff. Log error after exhaustion.

13. EXPLAINABILITY & AUDITABILITY

Principle: Every score must be traceable to specific signals. The dashboard should be able to show an analyst exactly *why* an email received its header risk score. This is achieved through three mechanisms:

Mechanism	Storage	Granularity
Rule hits	`rule_hits` table (PostgreSQL)	One row per fired deterministic signal (Block 1). Each row includes `rule_id`, `score_impact`, and `match_detail` (JSONB with specifics like "SPF result was 'fail'"). Queryable by email_id for full audit trail.
Sub-score breakdown	`scores.header` Kafka message → persisted in `emails.analysis_metadata` (JSONB)	Four sub-scores (auth, domain, typosquat, routing) are always preserved alongside the composite. The dashboard displays these as a stacked bar or breakdown chart.
SHAP values	`scores.header` Kafka message → `emails.analysis_metadata` (JSONB)	Block 5 ensemble outputs SHAP contributions per sub-score, showing the relative importance of each block's output to the final score. Example: "authentication failure contributed 45%, domain risk contributed 35%."

14. PERFORMANCE TARGETS

Metric	Target	Measurement
End-to-end latency (Kafka in → Kafka out)	< 4s p95 (cache hit), < 8s p95 (cache miss + enrichment)	Service histogram metric
Block 1 latency	< 5ms p99	Deterministic logic, no I/O
Block 2 latency (cache hit)	< 50ms p99	Redis get + ONNX inference
Block 2 latency (cache miss)	< 5s p95	WHOIS + DNS + GeoIP + ONNX
Block 3 latency	< 50ms p99	Embedding similarity (pre-loaded models)
Block 4 latency	< 10ms p99	Feature extraction + ONNX inference
Block 5 latency	< 15ms p99	7-feature meta-model + SHAP
Model accuracy (domain XGBoost)	F1 ≥ 0.95 on held-out test	Offline evaluation
Model accuracy (routing XGBoost)	F1 ≥ 0.93 on held-out test	Offline evaluation
Typosquatting precision	≥ 0.90 (low false positives)	Against curated domain-pair test set
Typosquatting recall	≥ 0.85	Against curated domain-pair test set

15. REFERENCES & RESEARCH SOURCES

Ref	Citation	Relevance to SVC-04
[1]	Jadhav, A. & Chandre, P.R. (2025). "A Hybrid Heuristic-Machine Learning Framework for Phishing Detection Using Multi-Domain Feature Analysis." ETASR, Vol. 15(5), pp. 27219–27226.	Hybrid ML + heuristic approach. RF achieves 99.85% on header features. XGBoost/CatBoost on URL features. Feature selection via RFE. Basis for hybrid Block 1 (deterministic) + Blocks 2–4 (ML) design.
[2]	Afonso, P., Maia, E., Amorim, I., & Praça, I. (2025). "MeAJOR Corpus: A Multi-Source	135K email dataset with engineered features. Identifies Received chain features, From/Return-Path discrepancies, SPF/DKIM/DMARC as high-

	Dataset for Phishing Email Detection." arXiv:2507.17978.	discriminative signals. XGB achieves 98.34% F1. Primary training data reference.
[3]	Lim, B., Huerta, R., Sotelo, A., Quintela, A., & Kumar, P. (2025). "EXPLICATE: Enhancing Phishing Detection through Explainable AI and LLM-Powered Interpretability." arXiv:2503.20796.	ML classifier with SHAP + LIME explainability achieving 98.4% accuracy. Basis for Block 5 SHAP-based explainability design.
[4]	Wang, Y. et al. (2025). "EvoMail: Self-Evolving Cognitive Agents for Adaptive Spam and Phishing Email Defense." arXiv:2509.21129.	Cross-modal fusion of text, headers, domains, and attachments. Identifies gaps in heterogeneous signal fusion. Supports multi-block architecture design.
[5]	Koide, T., Fukushi, N., Nakano, H., & Chiba, D. (2024). "ChatSpamDetector: Leveraging Large Language Models for Effective Phishing Email Detection." arXiv:2402.18093.	Header analysis methodology: Received chain validation, authentication result parsing, domain/IP correlation. Validates Block 4 feature set.
[6]	Piredda, P. et al. (2017). "Deepsquatting: Learning-Based Typosquatting Detection at Deeper Domain Levels." AI*IA 2017, LNCS 10640.	ML-based similarity measure for typosquatting via n-gram representations. Foundational work for Block 3 embedding approach.
[7]	Anvilogic (2024). "CE-Typosquat-Detect." HuggingFace model: Anvilogic/CE-Typosquat-Detect.	CrossEncoder model for domain pair typosquatting classification. Direct reference for Block 3 embedding method (M2).
[8]	TypoSmart (2025). "A Low False-Positive System for Detecting Malicious and Stealthy Typosquatting Threats." arXiv:2502.20528.	Embedding-driven name analysis + metadata verification reduces false positives by 70%. Validates Block 3 multi-method approach.
[9]	Hao, S. et al. (2016). "PREDATOR: Proactive Recognition and Elimination of Domain Abuse at Time-of-Registration." IEEE S&P.	Time-of-registration features predict malicious domains. Supports Block 2 WHOIS-based feature engineering (domain_age, lifespan, registrar patterns).
[10]	dnstwist — Domain name permutation engine. GitHub: elceef/dnstwist.	Open-source tool for generating typosquatting variants. Can be used to seed Block 3's training data for domain similarity models.
[11]	Uddin, M.A. & Sarker, I.H. (2024). "An Explainable Transformer-based Model for Phishing Email Detection." arXiv:2402.13871.	Fine-tuned DistilBERT for phishing detection with SHAP explanations. Validates transformer applicability at graduation project scale.
[12]	Tchouangang, E.C.N. et al. (2025). "Preventing Typosquatting with IDN Conversion: A Siamese Neural Network Approach." Springer.	Siamese networks for homograph detection. Supports Block 3 visual similarity method (M3).

16. IMPLEMENTATION PRIORITIES — PHASED ROLLOUT

Phase	Scope	Estimated Effort	Prerequisite
Phase 1 — Foundation	Block 1 (deterministic auth scoring) + Block 5 fallback (weighted average). This provides immediate improvement over "no scoring" with zero ML dependencies.	1–2 weeks	SVC-02 parser correctly extracts auth headers
Phase 2 — Domain Intel	Block 2 (enrichment pipeline + XGBoost domain model). Train initial domain model on PhishTank + Tranco data.	2–3 weeks	Phase 1 complete. WHOIS/DNS libraries integrated.
Phase 3 — Typosquatting	Block 3 (edit distance + embedding + homoglyph). Start with edit distance (M1) + homoglyph (M3); add embeddings (M2) as a second iteration.	2–3 weeks	Brand dictionary curated. Embedding model selected and tested.
Phase 4 — Routing Anomaly	Block 4 (routing XGBoost). Requires labeled header data for training.	2 weeks	MeAJOR or phishing_pot data available with full headers.

Phase 5 — Ensemble	Block 5 meta-model training. Replace weighted average with trained ensemble once sufficient production verdicts accumulate.	1 week	All blocks operational. Analyst verdicts provide training signal.

Total estimated effort: 8–11 weeks for a graduate team of 2–3 working with coding model assistance. Phase 1 is deployable independently and provides immediate value. Each subsequent phase is additive — the system degrades gracefully if later phases are deferred.

END OF DOCUMENT — SVC04-SPEC-v1.0 — CyberSiren Email Metadata Analysis Service — 2026-03-13

This document specifies the complete design of SVC-04 as a multi-block ML-inference microservice. It supersedes the heuristic-only Header Analysis Service defined in §3.4 of ARCH-SPEC-v2.1. All sub-block architectures, feature sets, model specifications, database access patterns, Kafka schemas, and phased implementation plan are defined herein.